

Explicando Decisões de LLMs via Otimização Inversa

Dissertação de Mestrado

George Gomes da Silva

Programa de Pós-Graduação em Informática

Abril de 2026

Problema: LLMs são caixas-pretas — tomam decisões sem explicar por quê.

Pergunta central da dissertação:

LLMs possuem um processo decisório estável e aprendível?

Relevância:

- **Explainability** — auditoria de decisões automatizadas.
- **Confiabilidade** — mesmo critério em situações diferentes?
- **Transferência de conhecimento** — se há um critério aprendível, ele pode ser extraído, reutilizado e comparado.
- **Regulação** — IA em áreas críticas (saúde, justiça, crédito) exige critérios versátil e inspecionáveis.

- H1 — Consistência.** O LLM mantém o mesmo critério decisório implícito. A métrica $\hat{W}_{\text{LLM}}$ estimada prevê classificações nos Problemas B e C.
- H2 — Few-shot amplifica.** Exemplos rotulados pela métrica aumentam a concordância LLM–métrica.
- H3 — LLM como aprendiz.** O LLM consegue aprender o critério de um perito externo via exemplos few-shot.
- H4 — Exemplos difíceis.** Exemplos *hard* (baixa margem) são mais informativos que *easy*.
- H5 — Estabilidade.** Comportamento reprodutível entre execuções (múltiplas seeds e repetições).

Nota: H1–H3, H5 formuladas para confirmação; H4 testada como hipótese falsificável.

Intuição: se o LLM tem critérios estáveis, podemos aprender sua *função objetivo implícita* a partir das decisões observadas.

Métrica de Mahalanobis diagonal:

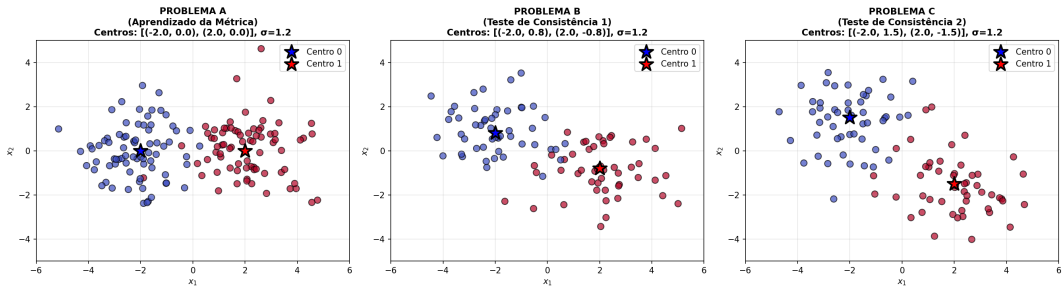
$$d_W(x, c) = \sum_{j=1}^d w_j \cdot (x_j - c_j)^2, \quad w_j \geq 0$$

Classificação nearest-centroid:

$$\hat{y}(x) = \arg \min_k d_W(x, c_k)$$

Otimização inversa: dado $\{(x_i, y_i^{\text{LLM}})\}$, inferir W tal que essas rotulações sejam ótimas sob d_W .
Framework de Ahuja & Orlin (2001).

4 problemas gerados com `sklearn.datasets.make_blobs`.

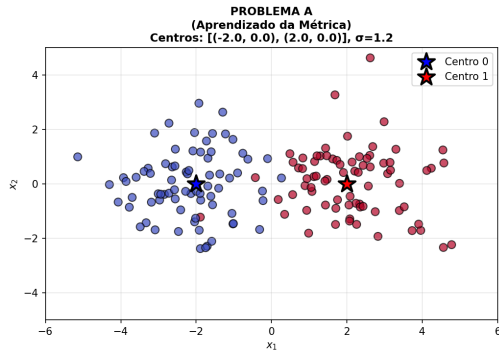


Problemas A (treino da métrica), B e C (testes de consistência). O Problema D (Fase D) tem perito externo com métrica conhecida.

Problema A — Treino da Métrica

Parâmetros:

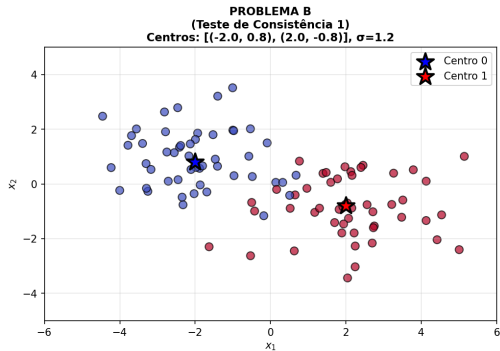
- $n = 150$ amostras, 2 classes
- Centróides: $c_0 = (-2, -0.5)$, $c_1 = (2, 0.5)$
- $\text{cluster_std} = 1.2$
- Seeds: $\{42, 123, 7\}$



Problema B — Teste de Consistência 1

Parâmetros:

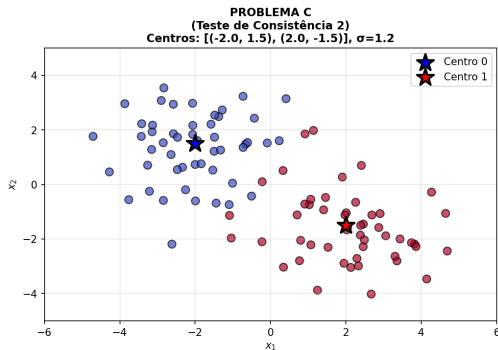
- $n = 100$ amostras, 2 classes
- Centróides deslocados *verticalmente* (geometria distinta de A)
- $\text{cluster_std} = 1.2$



Problema C — Teste de Consistência 2

Parâmetros:

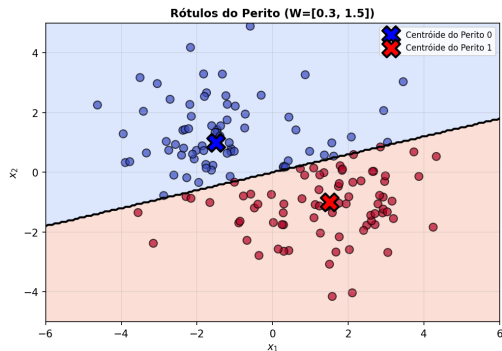
- $n = 100$ amostras
- Distorção geométrica *mais severa* (escala e orientação alteradas)
- `cluster_std` maior, separação não paralela aos eixos



Problema D — Perito Externo

Parâmetros:

- $n = 150$; perito rotula com $W_{\text{expert}} = [0.3, 1.5]$ (pondera x_2 5× mais que x_1)
- Centróides: $(-1.5, 1.0)$ e $(1.5, -1.0)$



Fronteira do perito (linha sólida) — LLM deve *aprender* esse critério via few-shot.

Configuração	Valor
Modelo LLM	gpt-4o-mini (OpenAI), temperatura 0.0
Seeds aleatórias	{42, 123, 7}
Repetições por config	3
Few-shot (Fases B/C)	{0, 5, 10, 20, 40}
Few-shot (Fase D)	{0, 5, 10, 20, 40}
Concorrência assíncrona	<code>asyncio.Semaphore(10)</code>
Nomes de classe (viés)	{A/B, 0/1, Positivo/Negativo, Azul/Vermelho} + inversos
Estratégias (Fase D)	{easy, hard, mixed, random}
Experts (Fase D)	{aniso_x2, aniso_x1, euclidean}
Variantes de prompt	{default, geometric, cot, tabular}
Total de chamadas ao LLM	114.478

Por que múltiplas seeds?

3 sementes (42, 123, 7) $\times$ **3 repetições** por configuração.

- ❶ **Estocasticidade dos dados:** `make_blobs` usa amostras aleatórias — uma única seed pode produzir geometria atípica.
- ❷ **Estocasticidade residual do LLM:** mesmo com temperatura 0, batching em GPU e top- k produzem variação entre chamadas.
- ❸ **Inferência estatística:** múltiplas seeds permitem
 - bootstrap 95% CI (10k reamostragens);
 - teste de reprodutibilidade (H5);
 - descartar achados dependentes de seed específica.
- ❹ **Prática padrão em ML empírico:** evita “p-hacking de seed” (Bouthillier et al., 2021).

Resultado esperado: direção de W (razão w_1/w_2) e fidelidade devem ser *qualitativamente estáveis* entre seeds.

Algoritmo 1 — Perceptron Estruturado (pseudo-código completo)

Formulação (Coelho, Borges & Fonseca Neto, CILAMCE 2017, Eq. 28):

$$\max_{\gamma} \gamma \quad \text{s.t.} \quad \underbrace{\sum_j w_j [(x_{ij} - c_{kj})^2 - (x_{ij} - c_{\ell j})^2]}_{\text{margem}_i(W)} + \lambda \alpha_i \geq \gamma \cdot \|w\|, \quad w \geq 0$$

Algoritmo (busca binária + perceptron interno):

- 1 Inicializar $w \leftarrow 1$, $\gamma_{\text{lo}} \leftarrow 0$, $\gamma_{\text{hi}} \leftarrow \infty$, $\gamma \leftarrow \gamma_{\text{init}}$.
- 2 Loop externo (busca binária em γ):
 - 1 Loop interno (perceptron multiplicativo) por max_iter épocas:
 - Para cada i com $\text{margem}_i(W) < \gamma$: $w \leftarrow w + \eta \cdot [(x_i - c_k)^2 - (x_i - c_{\ell})^2]$; projetar $w \geq 0$.
 - 2 Se violações = 0: salvar w^* ; $\gamma_{\text{lo}} \leftarrow \gamma$; $\gamma \leftarrow \gamma + \Delta\gamma$.
 - 3 Senão: $\gamma_{\text{hi}} \leftarrow \gamma$; $\gamma \leftarrow (\gamma_{\text{lo}} + \gamma_{\text{hi}})/2$ (bisseção); restaurar w^* .
 - 4 Parar quando $|\gamma_{\text{hi}} - \gamma_{\text{lo}}| \leq \text{tol}$.

Hiperparâmetros: $\eta = 1$ (padrão), $C = 10$, $\Delta\gamma = 0.05$, $\text{max_iter} = 50$, $\text{tol} = 10^{-4}$.

Algoritmo 2 — NNLS (Mínimos Quadrados, pseudo-código completo)

Formulação: resolver o sistema sobredeterminado $A w \approx b$ sob restrição de positividade.

$$\min_w \|A w - b\|^2 \quad \text{sujeito a} \quad w_j \geq 0$$

Pseudo-código:

- 1 Para cada ponto x_i com classe correta c_ℓ e rival c_k :
 - $A_i \leftarrow (x_i - c_k)^2 - (x_i - c_\ell)^2$ (vetor de tamanho d).
 - $b_i \leftarrow \text{target_margin} = 1$ (margem-alvo constante).
- 2 Empilhar $A \in \mathbb{R}^{n \times d}$ e $b \in \mathbb{R}^n$.
- 3 Chamar `w, residual = scipy.optimize.nnls(A, b)`.
- 4 (opcional) Normalizar $w \leftarrow w / \|w\|$ para visualização.

Garantia: $w \geq 0$ por construção (métrica diagonal semidefinida positiva). Ótimo único em problemas bem-condicionados (Lawson & Hanson, 1974).

Vantagem: zero hiperparâmetros $\Rightarrow$ resultado totalmente determinístico, independe de inicialização.

Justificativa de `target_margin = 1`: a razão w_1/w_2 (que determina a fronteira) é invariante à escala de b .

Pipeline de chamada ao LLM:

- 1 **Parser de 7 camadas** (`parse_llm_response`): extração por resposta limpa, `startswith`, `regex`, busca de palavra inteira, fuzzy matching, ...
- 2 **Retentativas**: `MAX_FORMAT_RETRIES` = 5 — se a resposta for malformada, reenvia o prompt com instrução reforçada.
- 3 **Fallback determinístico por hash MD5**: após 5 falhas, rótulo = $\text{MD5}(x_1, x_2) \bmod 2$. Substitui aritmética modular direta para não introduzir viés geométrico.
- 4 **Concorrência assíncrona**: `asyncio.Semaphore(10)` permite até 10 chamadas paralelas $\Rightarrow$ redução de $\sim 10\times$ no tempo de coleta.

Observação: temperatura 0.0 *não* garante determinismo (batching em GPU, top- k residual, hardware heterogêneo). Múltiplas seeds e repetições são **essenciais**.

Objetivo: capturar o processo decisório *intrínseco* do LLM.

Protocolo:

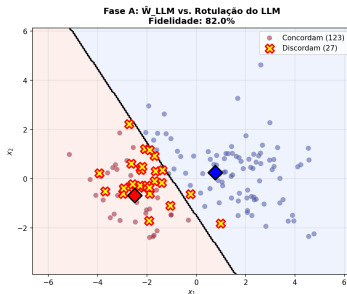
- 1 LLM classifica 150 pontos do Problema A **zero-shot** (sem exemplos).
- 2 Centróides $\hat{c}_k$ são recalculados a partir das decisões do LLM.
- 3 Cada algoritmo de otimização inversa (Perceptron, NNLS) estima um W independente a partir dessas decisões.
- 4 **Fidelidade:** % das decisões do LLM que a métrica aprendida reproduz.

Por que zero-shot? isola o *viés intrínseco* do modelo, sem contaminação de exemplos.

Fase A — $\hat{W}$ Aprendido e Fidelidade

Médias (Fase A, classes A/B, zero-shot, 3 seeds $\times$ 3 reps):

Algoritmo	$\hat{W}$ unitário típico (seed 42)	Razão w_1/w_2	Fid. Prob. A
Perceptron	[0.39, 0.92]	0.43	81.3%
NNLS	[0.20, 0.80]	0.25	84.7%



Mapa de erros da métrica (seed 42): pontos vermelhos = onde a métrica discorda do LLM.

Decomposição dos erros da métrica por distância à fronteira (seed 42):

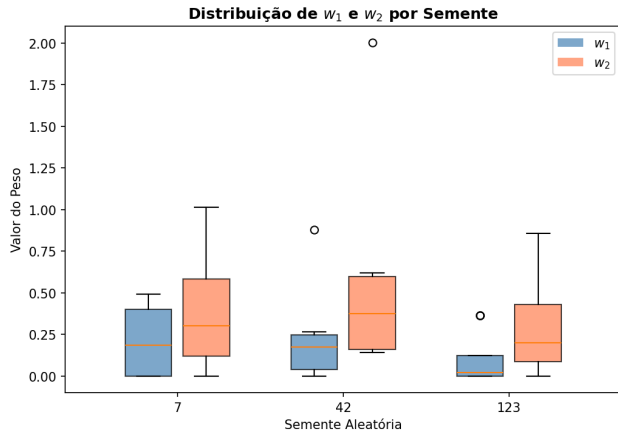
Região	N pontos	Erros	Taxa erro	% dos erros
Baixa margem (fronteira)	50	20	40.0%	74.1%
Margem média	49	6	12.2%	22.2%
Alta margem (longe)	51	1	2.0%	3.7%

Testes de significância (seed 42):

- Correlação ponto-bisserial (acerto $\times$ margem): $r = 0.395$, $p < 0.001$
- Mann-Whitney U (margem de acertos $>$ margem de erros): $U = 2708$, $p < 0.001$
- Margem média dos acertos: 4.51 vs. erros: 1.97

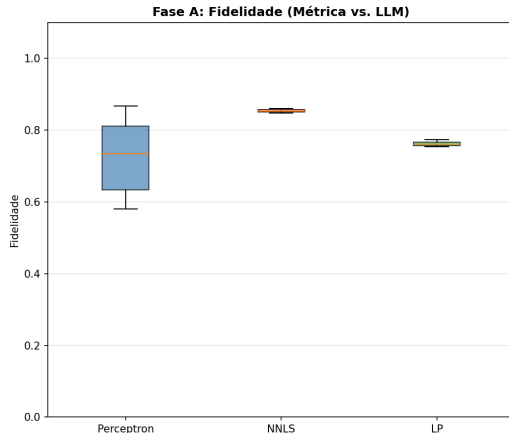
Resultado consistente entre seeds: seeds 7, 42, 123 todas confirmam que $\geq 65\%$ dos erros concentram-se próximos à fronteira.

Fase A — Distribuição de W entre Seeds



Direção de W estável: cosseno médio entre seeds = 0.984, range [0.975, 0.992]. **Razão w_1/w_2 :** média 0.405, CI_{95} [0.318, 0.458], CV = 32% em zero-shot, cai para 6% com few-shot ≥ 5 .

Fase A — Comparação dos 2 Algoritmos



Similaridade de cosseno entre algoritmos (zero-shot):

- Perceptron $\leftrightarrow$ NNLS: 0.986

Os 2 algoritmos convergem para métricas *geometricamente similares* — resultado robusto ao método de estimação.

Fase A — Sensibilidade a Hiperparâmetros do Perceptron

Re-execução do Perceptron com configs (seed 7), sem nova chamada ao LLM:

Variação	Valor	$\hat{W}$ unitário	cos(default)	Fid.
η (padrão 0.05)	0.01	[0.79, 0.61]	0.896	74.7%
	0.10	[0.35, 0.94]	0.996	82.7%
	0.50	[0.00, 1.00]	0.900	79.3%
C (padrão 10)	1.0	[0.23, 0.97]	0.977	86.0%
	50	[0.77, 0.64]	0.911	74.7%
δ_γ (padrão 0.05)	0.01	[0.36, 0.93]	0.996	82.7%
	0.10	[0.55, 0.84]	0.992	78.7%

Conclusão: $\cos \geq 0.86$ em todas as configs (3 seeds) $\Rightarrow W$ moderadamente sensível, mas a *direção* (que determina a fronteira) é estável. NNLS (sem hiperparâmetros) serve como referência.

Pergunta central: $\hat{W}_{\text{LLM}}$ (aprendido em A) generaliza para novos problemas?

Protocolo:

- 1 Aplicar $\hat{W}_{\text{LLM}}$ (da Fase A) e centróides locais dos Problemas B e C.
- 2 Coletar decisões do LLM nos mesmos pontos.
- 3 Comparar.

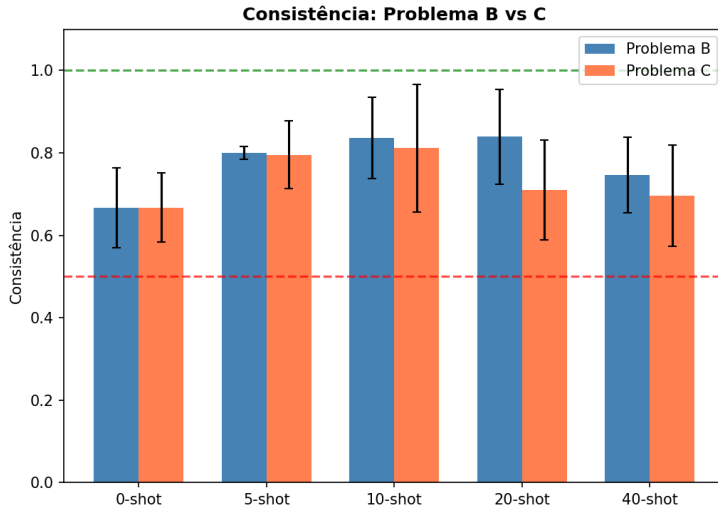
Métricas reportadas:

- Consistência (% concordância LLM vs. métrica)
- Kappa de Cohen (concordância corrigida pelo acaso)
- F1-Score (média ponderada por classe)

Consistência por n_shot (todas as variantes de classe, default prompt):

n_shot	Consist. B	Consist. C	Kappa B	Kappa C
0	66.7% $\pm$ 9.7%	66.7% $\pm$ 8.4%	0.189	0.215
5	80.0% $\pm$ 1.5%	79.5% $\pm$ 8.3%	0.598	0.570
10	83.7% $\pm$ 9.9%	81.2% $\pm$ 15.5%	0.629	0.614
20	83.9% $\pm$ 11.5%	71.0% $\pm$ 12.1%	0.671	0.413
40	74.6% $\pm$ 9.2%	69.6% $\pm$ 12.3%	0.492	0.395

Consistência média agregada: Problema B = 78.2% $\pm$ 12.1%; Problema C = 75.6% $\pm$ 14.5%.



Interpretação de Kappa (Landis & Koch, 1977):

- $[0.41, 0.60]$: concordância moderada — observada em zero-shot e $n \geq 20$
- $[0.61, 0.80]$: concordância substancial — observada em $n \in \{5, 10\}$

H1 (consistência): *parcialmente confirmada*. O LLM mantém W estável na *direção* entre seeds, mas a fidelidade da métrica no Problema A cai conforme a geometria de B/C se distancia de A.

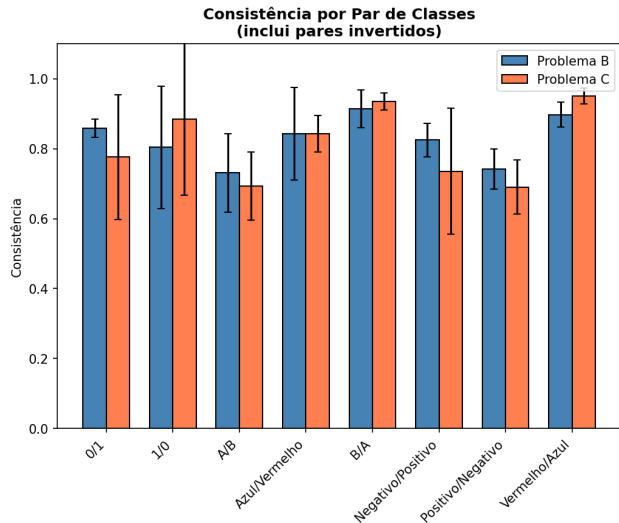
Observação curiosa: consistência *decai* para $n = 40$ em ambos problemas. Hipótese: com muitos exemplos, o LLM pode começar a interpolar entre eles em vez de seguir um critério consistente, aproximando-se do viés dos próprios exemplos.

Métrica estimada vs. Euclidiana (zero-shot, B): $\Delta = +14.8\%$, $CI_{95} [+3.5\%, +30.4\%] \Rightarrow$ **métrica significativamente melhor**.

Consistência média ($n = 10$) por ordenação de rótulos:

Ordem	Consist. B	Consist. C
A/B	81.0%	67.7%
B/A (inverso)	91.5%	93.5%
0/1	85.9%	77.6%
1/0 (inverso)	80.4%	88.5%
Azul/Vermelho	84.3%	84.3%
Vermelho/Azul (inverso)	89.8%	95.1%

Achado: inverter a ordem das classes no prompt **altera** o desempenho em até 25 p.p. ($A/B \rightarrow B/A$). O LLM tem *viés de posição* na enumeração de classes.



Efeito de nomes semânticos (zero-shot, 3 seeds):

Nomes das features	Fid. A	Consist. B
x1 / x2 (neutro)	72.8%	66.9%
altura / peso (semântico)	84.0%	74.0%
feature_1 / feature_2 (técnico)	58.0%	54.0%

Achado:

- Nomes semânticos (altura/peso) aumentam a fidelidade em **+11 p.p.**
- Nomes genéricos (feature_1/feature_2) **reduzem** a fidelidade em **-14 p.p.**
- O LLM está sensível à *semântica* do nome — não apenas à distribuição numérica.

4 variantes testadas nas Fases A-C em todas as seeds:

`default` Template padrão (baseline): coordenadas numéricas, pergunta direta.

`geometric` Contexto *geométrico* explícito: “*considere cada ponto em um plano 2D*”.

`cot` *Chain-of-thought*: instrui raciocinar passo a passo antes de responder.

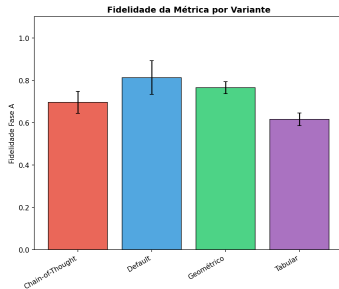
`tabular` Coordenadas apresentadas como tabela markdown.

Protocolo: Fases A-C completas em todas as seeds para cada variante. Se $\hat{W}$ mudar entre variantes $\Rightarrow$ o prompt *confunde a medição* (viés de formulação).

Variantes de Prompt (2/2) — Resultados

Fase A (zero-shot), média sobre 3 seeds:

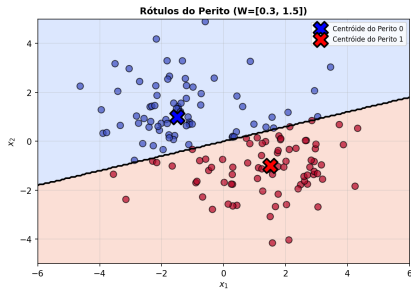
Variante	Fid. A	Consist. B	Consist. C
default	81.3%	72.4%	70.6%
geometric	76.4%	~ 75%	73.8%
cot	69.6%	~ 68%	61.1%
tabular	61.6%	~ 60%	60.6%



default e *geometric* são equivalentes; *cot* degrada (raciocínio explícito introduz ruído); *tabular* é o pior (formato compete com a tarefa).

Inversão de papéis: agora o LLM é o *aprendiz*.

- Perito externo com $W_{\text{expert}} = [0.3, 1.5]$ rotula os 150 pontos do Problema D.
- Acurácia do perito vs. ground-truth: 89.9%.
- LLM recebe $n \in \{0, 5, 10, 20, 40\}$ exemplos rotulados pelo perito (few-shot).
- LLM tenta *aprender* o critério do perito e classificar o restante.



Linha sólida: fronteira do perito ($x_2 \approx 5 \times$ mais importante que x_1).

Fase D — Estratégias de Seleção de Exemplos

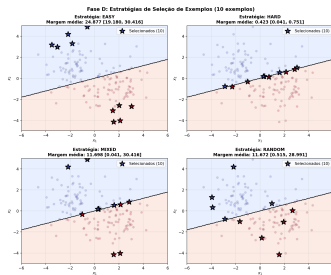
4 **estratégias** de seleção dos exemplos few-shot, balanceadas por classe:

easy Pontos com *alta margem* (longe da fronteira) — prototípicos.

hard Pontos com *baixa margem* (próximos da fronteira) — ambíguos.

mixed 50% easy + 50% hard.

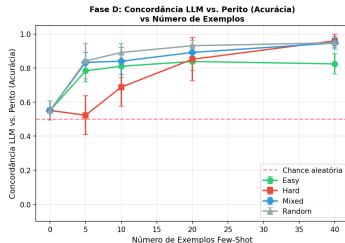
random Seleção aleatória balanceada (baseline sem estratégia).



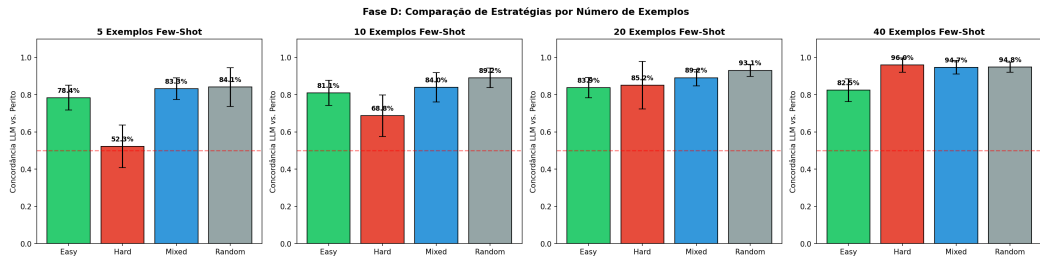
Fase D — Curvas de Aprendizado

Acurácia média LLM vs. perito (expert = aniso_x2, 3 seeds, 3 reps):

n_shot	easy	hard	mixed	random
0	60.9%	60.9%	60.9%	60.9%
5	85.8%	62.5%	82.0%	84.9%
10	86.9%	67.4%	79.4%	88.2%
20	90.2%	84.7%	86.9%	92.1%
40	88.0%	97.7%	96.6%	94.0%



Fase D — Comparação de Estratégias



Melhor estratégia por `n_shot`: 5-shot = **random** (84.1%); 10-shot = **random** (89.2%); 20-shot = **random** (93.1%); 40-shot = **hard** (96.0%).

Refutação de H4 — Por que “easy” vence “hard”?

Acurácia média por estratégia (todos $n > 0$):

Estratégia	Acurácia média
random	90.3%
mixed	87.8%
easy	81.5%
hard	75.6%

Resultado: H4 **refutada**. *Exemplos easy (81.5%) superam hard (75.6%) consistentemente.*

Interpretação proposta:

- Exemplos próximos à fronteira são *ambíguos* — não fornecem padrões claros de cada classe ao LLM.
- Exemplos fáceis funcionam como *âncoras prototípicas*, definindo bem as regiões de cada classe.
- Diferença conceitual com active learning em redes treinadas por gradiente: lá, exemplos difíceis são mais informativos *porque alteram os pesos*. Em LLMs com few-shot, o “aprendizado” é condicionamento, não atualização.

Acurácia média do LLM vs. perito (média sobre todas estratégias e n 's, exceto zero-shot):

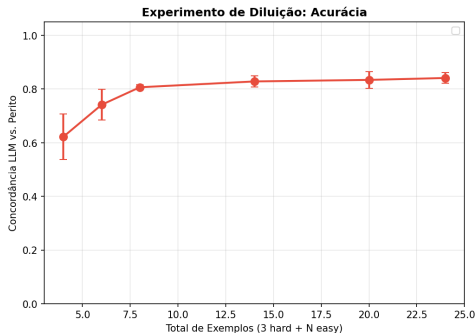
Expert	W_{expert}	Acurácia média
aniso_x2 (x_2 dominante)	[0.3, 1.5]	85.4% $\pm$ 10.9%
aniso_x1 (x_1 dominante)	[1.5, 0.3]	83.4% $\pm$ 13.9%
euclidean (pesos iguais)	[1.0, 1.0]	82.5% $\pm$ 13.4%

Interpretação: o LLM consegue aprender os 3 critérios geométricos distintos com desempenho similar $\Rightarrow$ a capacidade de aprendizado **não depende** da anisotropia específica do perito. Evidência forte para H3.

Experimento de Diluição

Design: 4 exemplos *hard* fixos + n exemplos *easy* progressivos ($n \in \{0, 2, 4, 10, 16, 20\}$).

Pergunta: os fáceis “diluem” os difíceis e a performance deteriora?



Achado: performance **aumenta monotonicamente** com mais easy (57% $\rightarrow$ 86%). Não há diluição — pelo contrário, os easy *compensam* a ambiguidade dos hard. Consistente com a refutação de H4.

4 ordenações dos mesmos exemplos (estratégia mixed):

Ordenação	5-shot	10-shot	20-shot
alternating	82.0%	85.1%	87.3%
class0_first	82.0%	80.0%	86.2%
class1_first	78.5%	83.1%	85.6%
shuffled	80.7%	87.3%	85.6%

Testes de Kruskal-Wallis por n_shot :

- 5-shot: $H = 0.560$, $p = 0.905$ — não significativo
- 10-shot: $H = 4.741$, $p = 0.192$ — não significativo
- 20-shot: $H = 1.171$, $p = 0.760$ — não significativo

Resultado: variação total < 3 p.p. $\Rightarrow$ **recency bias negligenciável** na faixa testada.

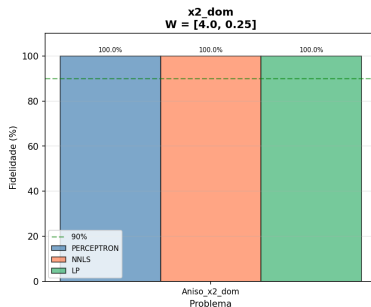
Sanity check: gerar dados sintéticos com W_{gt} conhecido, rotular com W_{gt} , e verificar se os 3 algoritmos recuperam W_{gt} .

Configs testadas (seed 42, Problema D):

Config	W_{gt}	Perceptron $\cos(W_{\text{rec}}, W_{\text{gt}})$	NNLS cos
aniso_x2	[0.3, 1.5]	0.9999	0.9933
aniso_x1	[1.5, 0.3]	0.9979	0.9894
euclidean	[1.0, 1.0]	0.9990	0.9960
forte_aniso	[11.1, 0.44]	0.9999	0.9999

Resultado: os 2 algoritmos recuperam W_{gt} com $\cos \geq 0.98$ em todas as configs anisotrópicas $\Rightarrow$ **piso de sanidade confirmado**. Falhas em dados reais do LLM não são falhas dos algoritmos.

Experimento: aprender W em uma geometria (x_2_dom) e aplicar em outras (x_1_dom , *forte_aniso*).



Achado: quando a geometria muda, a consistência **cai drasticamente** — W “aprendido” em uma geometria *não é universal*. A métrica é *acoplada* aos centróides do problema. Implicação para H1: inconsistência cross-problem reflete desalinhamento geométrico, não necessariamente variabilidade do LLM.

Motivação: a métrica diagonal é limitada — não captura interações entre features.

Solução: adicionar uma *terceira feature* $x_3 = x_1 \cdot x_2$ (kernel quadrático implícito).

Nova métrica:

$$d_W(x, c) = w_1(x_1 - c_1)^2 + w_2(x_2 - c_2)^2 + w_3(x_3 - c_3)^2$$

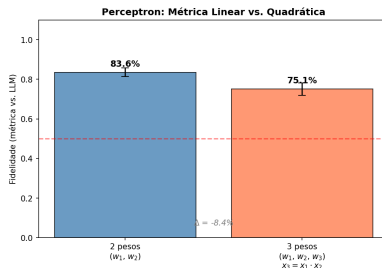
Protocolo de comparação:

- 1 LLM classifica com 2 features (baseline).
- 2 LLM classifica com 3 features (agora x_1, x_2, x_3).
- 3 Cada um dos 3 algoritmos aprende $W \in \mathbb{R}^3$.
- 4 Reportar fidelidade e peso de x_3 — se $w_3 > 0$, a interação importa.

Projeção R3 — Resultados

Fidelidade (3 seeds, média):

Condição	Algoritmo	Fid.	$\hat{W}$	Peso de x_3
LLM 2 features	—	84.4%	—	—
LLM 3 features + Perceptron	Perceptron	75.1%	[0.2, 0.6, 0.1]	~ 8%
LLM 3 features + NNLS	NNLS	86.2%	[0.04, 0.15, 0.02]	~ 8%



Achado: adicionar x_3 **não melhora** significativamente a fidelidade $\Rightarrow$ LLM não captura interação quadrática neste problema. Métrica diagonal linear é suficiente.

3 classificadores treinados nos **mesmos exemplos few-shot** da Fase D e avaliados no **mesmo conjunto de teste**:

k-NN `sklearn.KNeighborsClassifier`, $k = \min(5, n_shot)$, ajustado ímpar.

Logistic Regression regularização L2, `max_iter = 1000`.

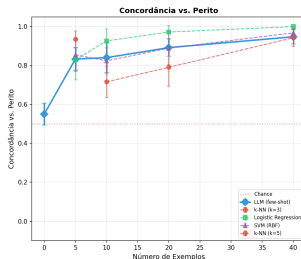
SVM (RBF) `sklearn.SVC(kernel='rbf')`, parâmetros padrão.

Pergunta: o LLM faz algo que um classificador trivial *não* faria com os mesmos dados? Se baselines superam o LLM no few-shot $\Rightarrow$ o LLM não está agregando informação.

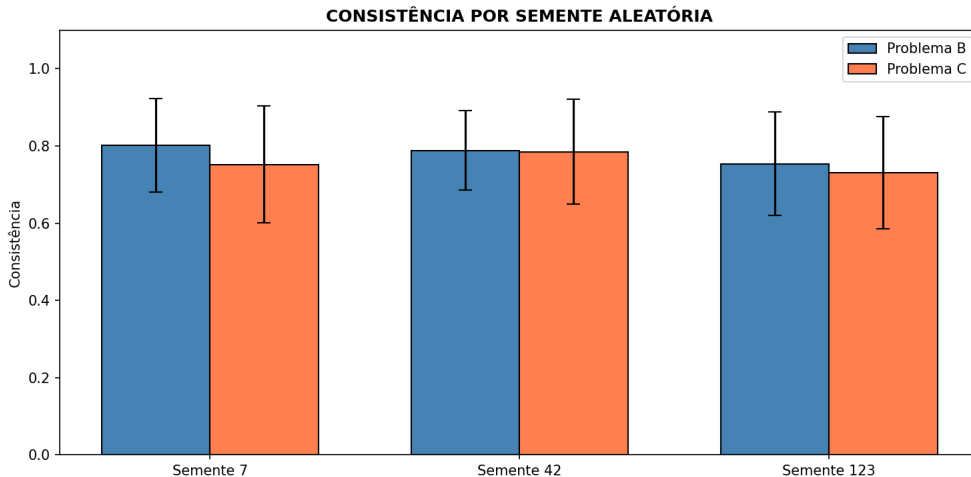
Baselines Clássicos — Resultados

Acurácia média vs. perito (todos n_shot , todas estratégias):

Modelo	Acurácia média	Kappa
LLM (melhor config)	$\approx 96\%$	≈ 0.92
Logistic Regression	93.4%	0.868
SVM (RBF)	87.5%	0.750
k-NN ($k = 5$)	89.5%	0.790
k-NN ($k = 3$)	84.0%	0.684



Conclusão: LLM *não é dominado* por baselines; Logistic Regression é o mais próximo. A vantagem do LLM está em robustez semântica, não em acurácia pura.



Consistência entre seeds (agregada): média 76.9%, desvio $\pm 13\%$; cosseno médio entre W s = 0.984. H5 sustentada — comportamento reprodutível.

Bootstrap 95% CI (10k reamostras), Fases A-C (classes A/B, default prompt):

Métrica (n_shot)	Média	CI ₉₅
Fidelidade A (Perc, $n = 0$)	0.813	[0.762, 0.845]
Consistência B (Perc, $n = 0$)	0.724	[0.677, 0.765]
Consistência B (Perc, $n = 10$)	0.810	[0.770, 0.851]
Kappa B (Perc, $n = 10$)	0.612	[0.536, 0.689]
Kappa C (NNLS, $n = 20$)	0.530	[0.393, 0.666]

Teste: Zero-shot vs. 10-shot (Wilcoxon pareado, Cohen's d):

- Consist. B: $\Delta = +7.7$ p.p., CI₉₅ [+6.1, +10.0], $d = 1.29$ (*grande*)
- Consist. C: $\Delta = -3.6$ p.p., CI₉₅ [-15.9, +3.7], $d = -0.72$ (*médio*)

Teste: Métrica vs. Euclidiana (zero-shot, B): $\Delta = +14.8$ p.p., CI₉₅ [+3.5, +30.4] $\Rightarrow$ significativo.

Testes não-paramétricos adicionais:

1. Correlação ponto-bisserial (acerto do LLM $\times$ margem do ponto, Fase A):

- Seed 42: $r = 0.395$, $p < 0.001$
- Seed 7: $r = 0.438$, $p < 0.001$
- Seed 123: $r = 0.232$, $p = 0.004$

$\Rightarrow$ *Correlação positiva significativa entre margem e acerto em todas as seeds.*

2. Mann-Whitney U (margem de acertos $>$ margem de erros):

- Seed 42: $U = 2708$, $p < 0.001$
- Seed 7: $U = 2806$, $p < 0.001$
- Seed 123: $U = 1834$, $p = 0.002$

$\Rightarrow$ *Erros concentram-se em pontos com baixa margem, confirmado em todas as seeds.*

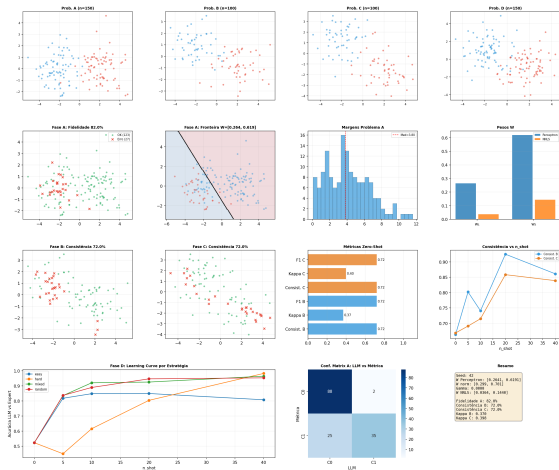
3. Kruskal-Wallis (ordem dos exemplos few-shot):

- 5-shot: $H = 0.56$, $p = 0.91$; 10-shot: $H = 4.74$, $p = 0.19$; 20-shot: $H = 1.17$, $p = 0.76$.

$\Rightarrow$ *Ordem dos exemplos NÃO afeta performance (recency bias ausente).*

Dashboards Exploratórios por Seed

Dashboard Completo do Experimento -- Seed 42



Dashboard consolidado (seed 42): distribuição de W , matrizes de confusão, curvas de aprendizado e comparação entre algoritmos em uma única vista.

Hip.	Descrição	Evidência	Veredito
H1	Consistência	Consist. B = 78%, C = 76%; Kappa em [0.4, 0.7]	Parcial
H2	Few-shot amplifica	$\Delta = +7.7$ p.p., $d = 1.29$ (zero vs 10-shot, B)	Confirmada
H3	LLM como aprendiz	Fase D: 55% $\rightarrow$ 96% com 40-shot (qualquer expert)	Confirmada
H4	Hard mais informativo	Easy (82%) > Hard (76%); random = 90%	Refutada
H5	Estabilidade	cos entre seeds = 0.984; CV da razão = 6% em $n \geq 5$	Confirmada

Destaque: H4 refutada é o achado mais original — contradiz a intuição de active learning e sugere que few-shot em LLMs funciona por *condicionamento prototípico*, não por *atualização de pesos*.

Prompts literais (1/2) — zero-shot e few-shot default

Prompt zero-shot (default):

You are a binary classifier for 2D points.

Classify the given point as Class A or Class B.
Answer ONLY with "A" or "B", nothing else.

Point to classify:

x1 = -1.4321

x2 = 0.8932

Your classification:

Prompt few-shot (default, 5 exemplos):

You are a binary classifier for 2D points.

Learn the classification pattern from the examples below, then classify the new point.

Examples:

x1 = -1.5000, x2 = 1.0000 -> A

x1 = 1.5000, x2 = -1.0000 -> B

x1 = -1.2000, x2 = 0.5000 -> A

x1 = 1.0000, x2 = -0.7000 -> B

x1 = -0.8000, x2 = 0.9000 -> A

Answer ONLY with "A" or "B", nothing else.

Point to classify:

x1 = -1.4321

x2 = 0.8932

Your classification:

Prompts literais (2/2) — variantes geometric / CoT / tabular

Geometric: adiciona contexto espacial ao default

You are a binary classifier for 2D points represented as (x1, x2) coordinates in a plane. The boundary between classes is geometric. Classify ... [resto igual ao default]

Chain-of-thought (CoT): pede raciocínio passo a passo

You are a binary classifier for 2D points. Think step by step about which class is closer based on the examples, then answer.

Examples: ...
Point: x1=-1.4321, x2=0.8932

Reasoning (be brief), then final answer "A" or "B":

Tabular: coordenadas como tabela markdown

x1	x2	class
-1.5	1.0	A
1.5	-1.0	B
...		

Classify | -1.4321 | 0.8932 | ? |

- Os papéis se invertem: um *perito* (expert ou métrica oráculo) rotula os dados; o LLM tenta *reproduzir* essas decisões via few-shot.
- Cobre Fase D (sintético anisotrópico), peso $\times$ altura (real, fronteira elíptica), meia-lua (sintético não-linear).
- Hipóteses ligadas a este bloco:
 - **H3** — LLM é capaz de aprender critério externo via few-shot.
 - **H4** — exemplos difíceis ensinam mais (**REFUTADA**).
 - **H5** — estabilidade entre seeds (transversal).

Estudo de caso real — peso $\times$ altura (homem/mulher)

Base enviada pelo orientador (e-mail 30/04/2026 19:15):

- ~ 100 amostras normalizadas em $[-1, 1]$, balanceado 50/50.
- Classificador ótimo bayesiano: *ellipse*

$$f(x_1, x_2) = x_2^2 - x_2 + x_1^2 - x_1 + \text{cte}$$

(fronteira quadrática).

Protocolo (e-mail 22:04):

- Repetir apenas Fases A e D nesta base.
- Variantes de prompt: (x_1, x_2) neutro vs. (peso, altura) semântico.
- Pergunta: classificar como **Homem** ou **Mulher**.
- Fase A com 2 / 3 / 4 features:
 - 2: (x_1, x_2)
 - 3 (hipérbole): $(x_1, x_2, x_1 \cdot x_2)$
 - 4 (ellipse): (x_1, x_2, x_1^2, x_2^2)
- Reportar fidelidade vs. LLM e acurácia vs. rótulo real.
- Identificar melhor n -features e rodar Fase D com essa métrica.

Resultados numéricos serão preenchidos após a próxima execução com `RUN_HOMEM_MULHER=True`.

Problema E — meia-lua (sintético não-linear)

Justificativa (reunião ~2558s): o experimento R3 em A/B/C *lineares* não melhorou ao adicionar $x_1 \cdot x_2$ (84% $\rightarrow$ 75%). Aqui sim — a meia-lua é *genuinamente não-linear*.

Setup:

- `sklearn.datasets.make_moons(n_samples=150, noise=0.15)`.
- Pergunta: classificar como classe A ou B.
- Fase A com 2 / 3 / 4 features (mesmas configurações da Parte 2).
- Fase D com a melhor métrica.

Expectativa (orientador ~2740s):

“Se o problema é linear, matematicamente ele não vai melhorar [...] Agora se o problema é mais difícil, de repente se com a outra componente a gente conseguir melhorar a métrica, é sinal de que ela capturou uma não linearidade na estratégia do LLM.”

Resultados numéricos serão preenchidos após a próxima execução com `RUN_PROBLEM_E=True`.

Tabela consolidada de ganhos ao passar de 2 → 3 → 4 features:

Problema	Tipo	Ganho 2→3 feat.	Ganho 2→4 feat.
A / B / C (gaussianas)	Linear	~ 0% (referência atual)	—
Meia-lua	Não-linear	a preencher	a preencher
Peso × altura	Não-linear (elipse)	a preencher	a preencher

Hipótese de trabalho: ganho apenas em problemas não-lineares. Em problemas lineares, a métrica diagonal 2D já representa a fronteira — adicionar features quadráticas não ajuda. Em problemas não-lineares, features quadráticas são necessárias para a métrica diagonal capturar a fronteira.

Citação orientador (~2843s): *“Eu acho que a gente fecha até um artigo já”.*

Obrigado!

Perguntas?

George Gomes da Silva
georgegomesdasil@gmail.com